A Worked Case: Trust Gates on Pull Request #41852

`Azure/azure-sdk-for-python`, authored by the GitHub Copilot coding agent

Supplement to *Engineering Trust When AI Writes Your Software*

Evidence gathered 2026-09-28/29; all commands reproducible with the replication package

Abstract

This supplement walks through one pull request from the case study in full: what the reviewer saw, how the requirements were classified into trust hypotheses, what each of the four trust gates was asked to establish, what it produced, how the findings were triaged, and how the project's own later history confirmed the result. PR #41852 was chosen because its story closes without our intervention: seventeen days after the agent's change was merged with CI green and a human approval, a maintainer opened a fix for exactly the behaviour the claim check flags. Every output quoted here is in the replication package's `results/evidence/` folder, and every step can be re-run with `run_experiments.py`.

1 The pull request as the reviewer saw it

The task. A user reported that merely importing the package printed a line:

```
>>> import azure.ai.evaluation
[INFO] Could not import AIAgentConverter. Please install the dependency with `pip
    install azure-ai-projects`.
```

The line appeared for everyone, including users who never touch the optional converter classes. The GitHub Copilot coding agent was assigned the issue.

The description. The agent's PR body states the problem, then the solution: "Implemented lazy loading mechanism using Python's module-level `__getattr__` to defer imports until the classes are actually accessed", with a before/after summary ("**After**: Messages are only shown when someone actually tries to use the converter classes") and five numbered changes: removed the import-time print statements; added lazy import functions; implemented module-level `__getattr__`; applied the pattern to both `AIAgentConverter` and `SKAgentConverter`; added "comprehensive" tests.

The diff. In `azure/ai/evaluation/__init__.py` (+59/−12) the two `try: import ...except ImportError: print(...)` blocks are replaced by a factory and a module-level `__getattr__`:

```
_lazy_imports = {}

def _create_lazy_import(class_name, module_path, dependency_name):
    def lazy_import():
        try:
            module = __import__(module_path, fromlist=[class_name])
            cls = getattr(module, class_name)
            _patch_all.append(class_name)
            return cls
        except ImportError:
            raise ImportError(
```

```
                f"Could not import {class_name}. Please install the dependency with `
    pip install {dependency_name}`."
            )
    return lazy_import

_lazy_imports["AIAgentConverter"] = _create_lazy_import(
    "AIAgentConverter", "azure.ai.evaluation._converters._ai_services", "azure-ai-
    projects")
_lazy_imports["SKAgentConverter"] = _create_lazy_import(
    "SKAgentConverter", "azure.ai.evaluation._converters._sk_services", "semantic-
    kernel")

__all__ = [
    "evaluate",
    ...
    # Include lazy imports in __all__ so they appear as available
    "AIAgentConverter",
    "SKAgentConverter",
]

def __getattr__(name):
    """Handle lazy imports for optional dependencies."""
    if name in _lazy_imports:
        return _lazy_imports[name]()
    raise AttributeError(f"module '{__name__}' has no attribute '{name}'")
```

Two lines matter more than the rest and are easy to pass over: the two names are added to `__all__`, with the agent's own comment, "so they appear as available". A second file, `red_team/__init__.py`, changes a `print` on missing `pyrit` into a `raise ImportError` (4 lines). A new test file, `tests/unittests/test_l` (135 lines), holds three tests.

The merge. Squash commit `d76b8ba34`, merged 2025-07-14 into `main`. CI reported success. One human maintainer approved. Nothing in the diff, the description, or the test results gave a reviewer a reason to ask a question.

2 Requirements and trust hypotheses

The framework asks the team to classify, before reading the implementation, what can be defended about each requirement the change is meant to satisfy. The requirements were extracted from the issue and the PR description only.

R1. Nothing is printed at import time when an optional dependency is absent.

R2. The package's public import surface remains usable for users who do not have the optional dependency.

R1 is the requirement the issue states. R2 is implicit: it is what "the fix must not break" and is the kind of requirement that never gets written down, which is exactly why the framework asks for it to be classified.

2.1 What the three levels demand

The classification is not a priority ranking; it is a commitment about evidence.

- MUST-HOLD: violation is unacceptable. The team owes the strongest evidence it can produce, and if the property cannot be defended, the requirement or the design must change.

- MAY-FAIL: occasional deviation is tolerable if it is detected and handled. The team owes detection, not proof.
- CANNOT-GUARANTEE: the team cannot promise the property. It owes an explicit statement of the limit and a containment strategy.

2.2 The two-question procedure, applied

Question 1: would a violation be a security, safety, data-integrity or compliance failure, or make the artifact unfit for its purpose? *Question 2:* does correct behaviour depend on resolving a specification ambiguity, on state the artifact cannot observe, on completeness over an open-ended input space, or on behaviour with no reference implementation? Both answers are recorded even when the first already decides the level.

	Q1	Q2	Level	Reasoning
R1	no	no	MAY-FAIL (H1)	A stray `[INFO]` line is noise, not a failure; the behaviour is fully specified and directly observable. One observation suffices as evidence.
R2	**yes**	**yes**	MUST-HOLD (H2)	A package that raises on import for users lacking an optional extra is unfit for that class of users (Q1). Whether R2 holds depends on what "using the package" means, and neither the issue nor the description says (Q2).

Why R2 is the interesting case. R2 answers yes to both questions. Under a priority scheme it is simply "high priority" and the reviewer moves on. Under this classification it is MUST-HOLD *and* ambiguity-dependent, which tells the team two things at once: strong evidence is required, and the evidence must cover the alternative readings of the ambiguity. The alternative readings are the standard ways Python code touches a package's names: `import x`, `from x import name`, `from x import *`, `hasattr(x, name)`, `dir(x)`, and documentation or IDE tooling that walks `__all__`. The agent resolved the ambiguity in favour of one reading, explicit attribute access, without saying so. This classification is what generated checks S1 and S2 in Section **??**; the causal chain from classification to detection is the point of the example.

Why R2 is not CANNOT-GUARANTEE. The ambiguity is resolvable: the import idioms are finite and known, so the property can be promised after enumerating them. CANNOT-GUARANTEE is reserved for properties the team could not defend even with effort. This is the judgment call in the PR, and raters may disagree on it; see the note at the end of this section.

2.3 Gate plan

The classification determines which gates serve which hypothesis:

Hypothesis	Level	Gates
H1 no output at import time	MAY-FAIL	G4 (one observation)
H2 import surface usable without the dependency	MUST-HOLD	G1, G4 (across import idioms); G2/G3 (is the mechanism defended at all?)
H0 referenced interfaces exist (standing, every PR)	MUST-HOLD	G1

> Both requirements were classified by a single analyst (the AI assistant) who had already seen the diff and the gate results; this classification is therefore not blind and is informed by hindsight. An independent human rater, working from the description only, extracted R1 but not R2. R2's double "yes" is the kind of case on which raters may differ (some would argue CANNOT-GUARANTEE); the human rater's blind classification supersedes this one. Part of the procedure's value is that it forces this disagreement into the open before the code is examined.

3 Environment and baseline

Everything below runs against the repository checked out at the merge commit, with one precondition that matters: the optional dependency must be *absent*, because the defect only exists in that state.

```
git checkout d76b8ba34
export PYTHONPATH=$PWD/sdk/evaluation/azure-ai-evaluation   # package used in place
export AZURE_TEST_RUN_LIVE=true AZURE_SKIP_LIVE_RECORDING=true   # neutralise the test-
    proxy fixture
python -c "import azure.ai.projects"   # -> ModuleNotFoundError: the precondition
python -m pytest -q tests/unittests/test_lazy_imports.py tests/unittests/
    test_evaluators   # 41 passed
```

The baseline is green. This is the premise of the escape argument: whatever the gates find below was already in code that conventional practice had accepted.

4 G1: interface resolution

What it establishes. That the change does not reference anything that does not exist (the "hallucinated interface" failure mode). It runs `pyflakes` on each changed file and keeps undefined-name reports on changed lines; resolves every import on a changed line with `importlib`; and checks every `module.attr` on a changed line against the real imported module.

What it exercised. 49 changed lines across the two `__init__.py` files; every import resolved; every attribute reference resolved.

Result. Pass, no findings.

Reading the result. A pass is evidence that one failure mode is absent, not evidence that the change is correct. The record says so in its `not_evaluated` field: method calls on objects of inferred type, string-referenced module paths (the `__import__` calls above take strings, which G1 cannot resolve statically), and behaviour with the dependency installed. G1 contributes nothing to H2 beyond ruling out one way of failing it; its value in this PR is that it ran and is on record.

5 G2 and G3: mutation on the changed lines

What they establish. Whether the tests that exist would notice if the changed code were wrong. G2 runs each mutant against the package's test suite; G3 runs the same mutants against only the tests the PR itself added. When the two agree, the PR's tests are the only tests exercising the change, which is the "closed loop" the paper's introduction describes.

Operators. Single-point mutants only on lines the PR added or modified: comparison flips, `and`/`or` swap, `not` removal, arithmetic swaps, constant changes (booleans, numbers, strings), `return X` → `return None`, `raise` → `pass`, and negation of `if` conditions.

Result. 20 mutants; 0 killed by the package suite; 0 killed by the PR's tests.

Line	Code	Mutation	Suite / PR tests
77	`return cls` (lazy_import)	`return None`	survived / survived
79	`raise ImportError(...)` (lazy_import)	remove `raise`	survived / survived
83	`return lazy_import`	`return None`	survived / survived
86–94	the two `_create_lazy_import(...)` calls	each of 8 string arguments mutated	survived / survived
139–140	`"AIAgentConverter"`, `"SKAgentConverter"` in `__all__`	string mutated	survived / survived
148	`if name in _lazy_imports:`	`in` → `not in`	survived / survived
148	same	negate condition	survived / survived
149	`return _lazy_imports[name]()`	`return None`	survived / survived
150	`raise AttributeError(...)`	remove `raise`	survived / survived
61, 147	two docstrings	string mutated	survived / survived (cosmetic)
11 (red_team)	`raise ImportError(...)`	remove `raise`	survived / survived

Discounting the two docstring mutants as cosmetic, eighteen behavioural mutants survive, including the ones that disable the mechanism outright: negating the `__getattr__` condition, returning `None` instead of the class, and deleting either `raise`.

Why: the tests test a copy. The PR's test file explains itself. Its first test opens with:

```
# Test imports that would normally fail with missing dependencies
# Since we can't easily control the dependency availability in the test environment,
# we test the lazy import setup directly

# This should not print any messages during setup
_lazy_imports = {}
_patch_all = []

def _create_lazy_import(class_name, module_path, dependency_name):
    ...   # a verbatim copy of the factory from __init__.py
```

and its second and third tests define a local function documented as "Mock `__getattr__` function like the one in `__init__.py`" and assert on that. The file never imports the mechanism it is named after. Its first test is also decorated `@pytest.mark.skipif(has_semantic_kernel, ...)`, so it is skipped precisely in the environments where the behaviour could be observed. The tests pass for any implementation of `__init__.py` whatsoever, which is what a 0/20 score means.

What G2/G3 contributed. Not the defect, but the question. A must-hold hypothesis (H2) whose implementing mechanism has no test protection at all cannot be defended by the evidence the PR ships, so the claim check must supply the evidence itself.

6 G4: claim checks

What it establishes. Whether each behavioural statement in the PR description is true of the code. Each claim becomes one executable check that would fail (or print a distinguishable result)

if the claimed behaviour were absent. The script runs at the merge commit and again with the changed files restored from the parent commit, so every check has a before/after value.

The claims, as sentences.

(a) No message is printed at import time.
(b) Messages are shown only when someone actually tries to use the converter classes.
(c) Module-level `__getattr__` intercepts attribute access.
(d) Both converters use the same pattern.
(e) The tests are comprehensive.

(e) is settled by Section 5. (a), (c) and (d) each map to one direct check. (b) is the claim that carries H2 and it is stated in terms of intent ("tries to use"), so the check is to find ways of *not* using the converters that still trigger the path. The two standard idioms that touch every name in `__all__` without using any of them are `from pkg import *` and `hasattr(pkg, name)`. These became checks S1 and S2; they are not mentioned anywhere in the PR, and they exist only because H2 was classified as ambiguity-dependent in Section **??**.

The five checks.

```
buf = io.StringIO(); sys.stdout = buf; import azure.ai.evaluation as ev; sys.stdout =
    old
print("C1 import-time output:", repr(buf.getvalue().strip()[:80]))
try: ev.AIAgentConverter; print("C2 attribute access -> resolved")
except ImportError as e: print("C2 attribute access -> ImportError:", str(e)[:60])
except AttributeError: print("C2 attribute access -> AttributeError")
try: print("S1 hasattr(module,'AIAgentConverter') ->", hasattr(ev, "AIAgentConverter"))
except Exception as e: print("S1 hasattr -> raises", type(e).__name__)
try: ns = {}; exec("from azure.ai.evaluation import *", ns); print("S2 import * -> ok")
except Exception as e: print("S2 import * -> raises", type(e).__name__)
print("S3 'AIAgentConverter' in __all__:", "AIAgentConverter" in ev.__all__)
```

Before and after. Run at the merge commit, then with the two `__init__.py` files checked out from `d76b8ba34~1`.

Check	Before the PR (parent)	After the PR (merge commit)	Claim
C1 stdout during import	`[INFO] Could not import AIAgentConverter...`	(nothing)	(a) supported
C2 `ev.AIAgentConverter`	`AttributeError`	`ImportError` with install hint	(c),(d) supported
S1 `hasattr(ev, "AIAgentConverter")`	`False`	**raises** `ImportError`	(b) not supported
S2 `from azure.ai.evaluation import *`	succeeds	**raises** `ImportError`	(b) not supported
S3 `"AIAgentConverter" in __all__`	`False`	`True`	the cause

Reading the result. The mechanism works as described for the reading of "use" the agent had in mind. It fails for two readings it did not have in mind, and the failure is not a message but an exception: any module that does `from azure.ai.evaluation import *` without the optional extra installed now cannot be imported at all, and `hasattr`, whose contract is to return `False` for absent attributes, raises instead. S3 identifies the single cause: the names are in `__all__`, so `import *` requests each of them, `__getattr__` is invoked, and the lazy import raises.

7 Triage

The triage rules are applied in order.

Confirmed defect? Yes. The behaviour at the merge commit contradicts the PR's own stated intent (claim (b)) and a standard-library contract (`hasattr`); it is reproduced by a script with a before/after comparison.

Escaped? Yes. CI was green at merge; no review comment mentions `__all__`, `import *` or `hasattr`; the defect is present in the merged code.

Failure mode. Assumption injection: the agent resolved "how is this package imported?" in favour of explicit attribute access and encoded that assumption in `__all__` without stating it.

Severity. Major: a user-visible failure (`import *` cannot be executed) for every user without the optional extra, on a MUST-HOLD hypothesis.

Second finding. Evidence gap: 0/20 mutants killed; the PR's tests test a copy of the mechanism and are skipped where the behaviour is observable.

Not evaluated. Behaviour with the dependency installed; the `red_team` change (a `print` turned into a `raise`, which makes `import azure.ai.evaluation.red_team` fail without `pyrit`; arguably a second instance of the same pattern, not examined).

> This triage was performed by one analyst and is recorded in `results/evidence/pr41852_G4.json`; an independent triage is required before it is cited.

8 Downstream confirmation

After triage, the repository's later history was searched for a fix. Two observations:

```
git show main:sdk/evaluation/azure-ai-evaluation/azure/ai/evaluation/__init__.py | grep
    -n '"AIAgentConverter"'
# names appear only inside the _create_lazy_import(...) calls; no longer in __all__
# __getattr__ and _lazy_imports are still present
```

Someone had removed exactly the two lines and nothing else. A GitHub search for merged PRs mentioning the converter found it:

> **PR #42313**, opened by a maintainer, merged 2025-07-31: "[evaluation] fix: Remove lazy imports from `__all__`". Description: "This pull request removes 'lazy imports' from `__all__` to unblock a release gate that tries to import everything from `__all__` (which fails since we raise an ImportError)."

Seventeen days elapsed between the agent's merge and the fix. The project's release gate is, functionally, check S2 executed late and expensively, at release time rather than at merge time. Nothing about the fix disputes the agent's change in any other respect; the mechanism stayed.

9 The claim–evidence–limitation statement

Claim	**Evidence**	**Limitations**
Importing `azure.ai.evaluation` no longer prints when the optional converters are unavailable, and direct attribute access to a converter raises an informative `ImportError`. *Scope:* explicit attribute access.	G4 C1 and C2, before/after. G1 pass (49 lines). 41 tests green at the merge commit.	`from azure.ai.evaluation import *` and `hasattr()` raise `ImportError` when the dependency is absent (confirmed defect; fixed downstream in #42313). The mechanism has no test protection: 0/20 mutants killed; the PR's tests exercise a copy. Behaviour with the dependency installed not evaluated. `red_team` import behaviour changed and not evaluated.

The scope line and the first limitation say the same thing from two sides, which is the correspondence rule: everything outside the claim's scope must appear as a limitation. Had this statement accompanied the PR, the reviewer would have seen "scope: explicit attribute access" next to a diff that adds names to `__all__`, and the question would have asked itself.

10 What each gate contributed, and what it cost

Gate	Contribution	Cost
G1	Ruled out hallucinated interfaces; on record	seconds
G2/G3	Showed the must-hold mechanism is undefended (0/20), which made G4 mandatory	≈1 min machine
G4	Found the defect (S1, S2) and its cause (S3)	≈25 min analyst
Downstream search	Confirmed it against the project's own history	≈10 min analyst

The counterfactual. Could conventional practice have caught this? The pipeline reported that the agent's tests passed, and they did: they pass for any `__init__.py`. Static analysis has no rule against putting a name in `__all__`. The reviewer read a fluent description whose "After" line answered the only question worth asking, and the diff comment "so they appear as available" reads as a feature. What found the defect was treating the description's claims as hypotheses, classifying the one that carried the risk as must-hold and ambiguity-dependent, and then enumerating the readings of the ambiguity. The project found it too, by waiting for a release gate to fail.

11 Reproduction

With the replication package (`replication/`) set up as described in its README:

```
python run_experiments.py run --prs 41852            # G1, G2/G3, G4; writes evidence/
    pr41852_*.json and pr41852_G4_output.txt
cat evidence/pr41852_G4_output.txt                     # the before/after table of
    Section 6
python run_experiments.py compare --prs 41852          # G1/G2/G3 records against
    results/evidence/
```

The precondition (`azure-ai-projects` absent) is checked by the runner before the PR is started.

Notes on method

Two points of honesty belong here. First, the earliest claim check written in this study (for a different PR, #44844) used a hypothetical call sequence and was replaced by the production caller's actual sequence before its finding was accepted; the discipline in Section **??** of deriving checks from the description and from real usage idioms, rather than from what seemed plausible, follows from that correction. Second, every judgment in Sections **??** and 7 is currently the assistant's; the human rater's classification and an independent triage are listed in the methodology document as prerequisites for citation.